

1. Introduction to GNN

2026-03-10, 16:08

Tags: [GNN Deep Learning](#)

1. Why Graphs

Types of Networks and Graphs

1. Networks (Natural Graphs)

Types of Networks and Graphs

- **Networks (also known as Natural Graphs):**
 - **Social networks:**
 - **Society** is a collection of 7+ billion individuals
 - **Communication and transactions:**
 - Electronic devices, phone calls, financial transactions
 - **Biomedicine:**
 - Interactions between **genes/proteins** regulate life
 - **Brain connections:**
 - Our **thoughts** are hidden in the connections between billions of neurons

2. Graphs (as a representation)

Types of Networks and Graphs

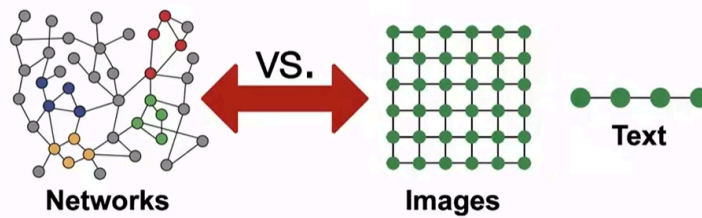
- **Graphs (as a representation):**
 - **Information/knowledge** are organized and linked
 - **Software** can be represented as a graph
 - **Similarity networks:** Connect similar data points
 - **Relational structures:** Molecules, Scene graphs, 3D shapes, Particle-based physics simulations

Main questions: How do we take advantage of relational structure for better prediction?

- The Complexity of Graph:

Networks are complex.

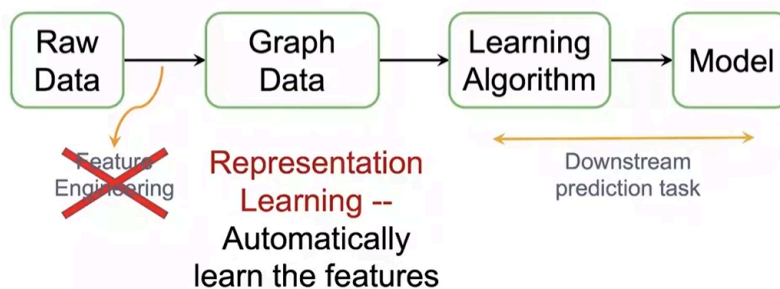
- Arbitrary size and complex topological structure (*i.e.*, no spatial locality like grids)



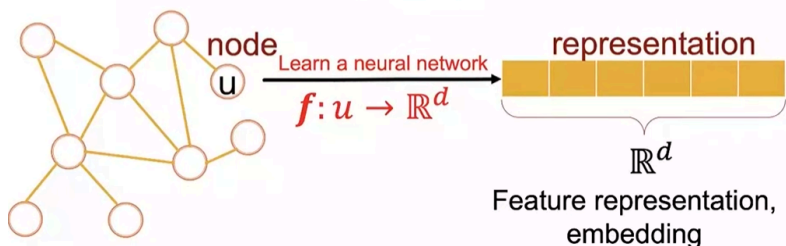
- No fixed node ordering or reference point
- Often dynamic and have multimodal features

- Representation Learning → the removal of feature engineering step

(Supervised) Machine Learning Lifecycle:
This feature, that feature. **Every single time!**



- Map nodes to d-dimensional embedding, such that similar nodes in the network are embedded close together.



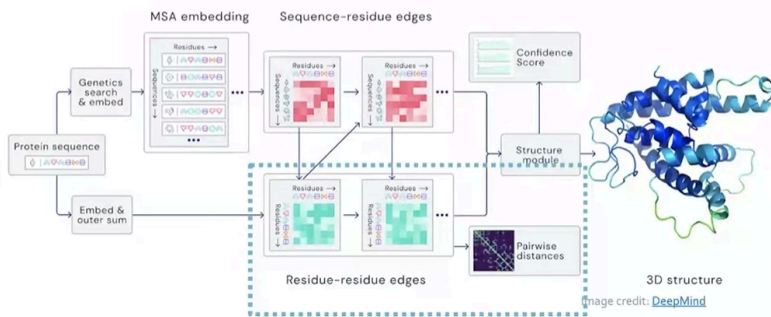
2. The Application of Graph Neural Network

Node-level Prediction

It is processing the specific label on the node
eg. Alpha Fold, representing **Spatial Graph**

■ Key idea: “Spatial graph”

- **Nodes:** Amino acids in a protein sequence
- **Edges:** Proximity between amino acids (residues)



Edge-level prediction

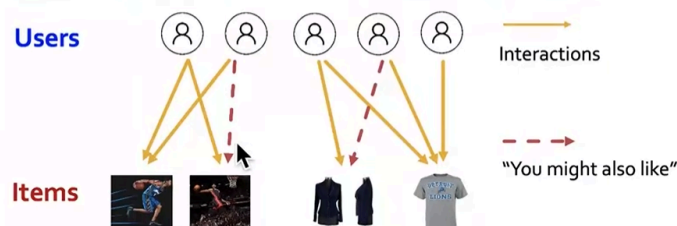
Edge predict the proximity of the existing linkage

- Recommender system (Whether a edge might exist between a consumer and a product)

■ Users interacts with items

- Watch movies, buy merchandise, listen to music
- **Nodes:** Users and items
- **Edges:** User-item interactions

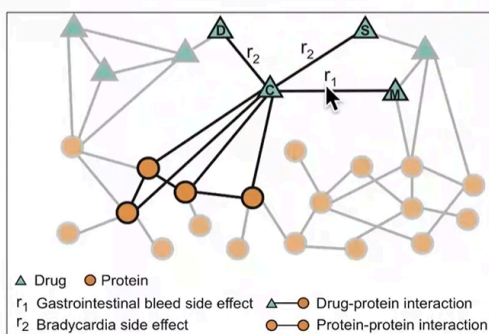
■ Goal: Recommend items users might like



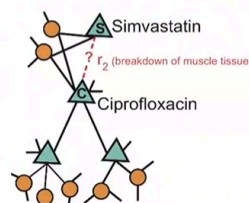
- Side effect prediction

Biomedical Graph Link Prediction

- **Nodes:** Drugs & Proteins
- **Edges:** Interactions



Query: How likely will Simvastatin and Ciprofloxacin, when taken together, break down muscle tissue?



Sub-graph level machine learning prediction

- The final location will be the node, and the traffic on the road will be the edge

Traffic Prediction via GNN

Predict via Graph Neural Networks

The diagram illustrates the model architecture for determining optimal routes and their travel time. It is divided into two main sections: a training phase and a prediction phase.

Training Phase (Left):

- Anonymised travel data** (represented by a teal circle with a location pin and a circular arrow icon) is **Analysed** to create **Supersegments** (represented by a teal circle with a road network icon).
- The **Supersegments** are used for **Training data** to train a **Graph neural network** (represented by a teal circle with a graph icon).

Prediction Phase (Right):

- The **Graph neural network** produces **Predictions** (indicated by a downward arrow).
- These predictions are used to rank routes, resulting in **Routes ranked by ETA** (represented by a red circle with a location pin and a road icon).
- The **Routes ranked by ETA** are then **Surfaced** (indicated by a rightward arrow) to the **Google Maps API** (represented by a blue circle with a location pin and a code icon) and the **Google Maps app** (represented by a blue circle with a location pin and a magnifying glass icon).
- The **Google Maps routing system** (represented by a purple circle with a location pin and a magnifying glass icon) also provides **Candidate user routes A+B** (indicated by an upward arrow) to the **Routes ranked by ETA**.

Used in Google Maps

THE MODEL ARCHITECTURE FOR DETERMINING OPTIMAL ROUTES AND THEIR TRAVEL TIME.

Image credit: [DeepMind](#)

- Used in Google Maps

THE MODEL ARCHITECTURE FOR DETERMINING OPTIMAL ROUTES AND THEIR TRAVEL TIME

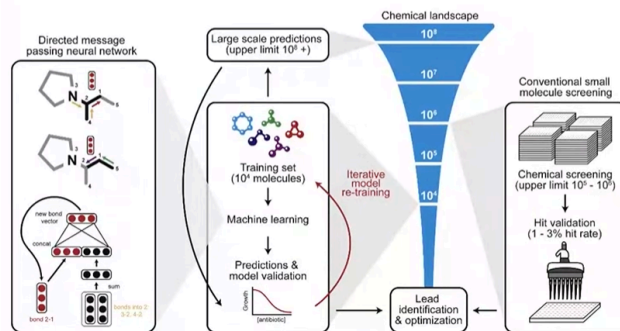
Image credit: [DeepMind](#)

Graph level machine learning prediction

- Find therapeutic effect of the molecules → graph classification model

Deep Learning for Antibiotic Discovery

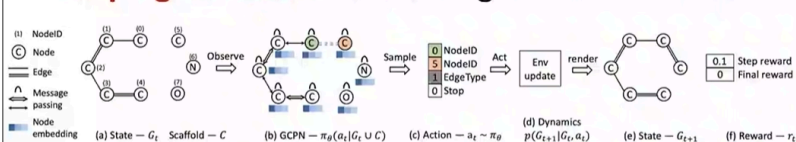
- A Graph Neural Network **graph classification model**
- Predict promising molecules from a pool of candidates



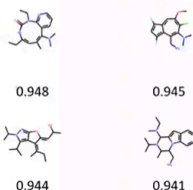
Stokes, Jonathan M., et al. "A deep learning approach to antibiotic discovery." *Cell* 180.4 (2020): 688-702.

- Graph generation: generate new structure and new molecules

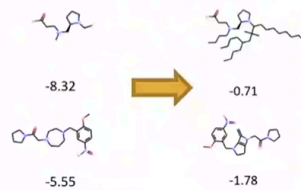
Graph generation: Generating novel molecules



Use case 1: Generate novel molecules with high drug likeness

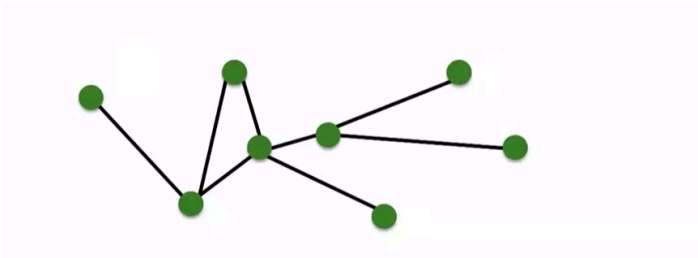


Use case 2: Optimize existing molecules to have desirable properties



3. Choice of Graph Representation

in graph, we have **objects, interaction, and system**



- **Objects:** nodes, vertices N
- **Interactions:** links, edges E
- **System:** network, graph $G(N,E)$

1. What are nodes?
2. What are edges?

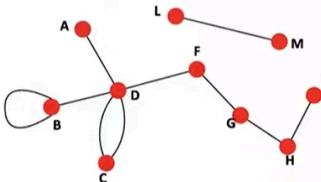
We need to choose proper network representation of a given domain/problem to determines our ability to use networks successfully.

Directed vs. Undirected Graphs

Simply is there an arrow or not...

Undirected

- **Links:** undirected (symmetrical, reciprocal)

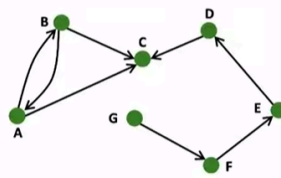


■ Examples:

- Collaborations
- Friendship on Facebook

Directed

- **Links:** directed (arcs)



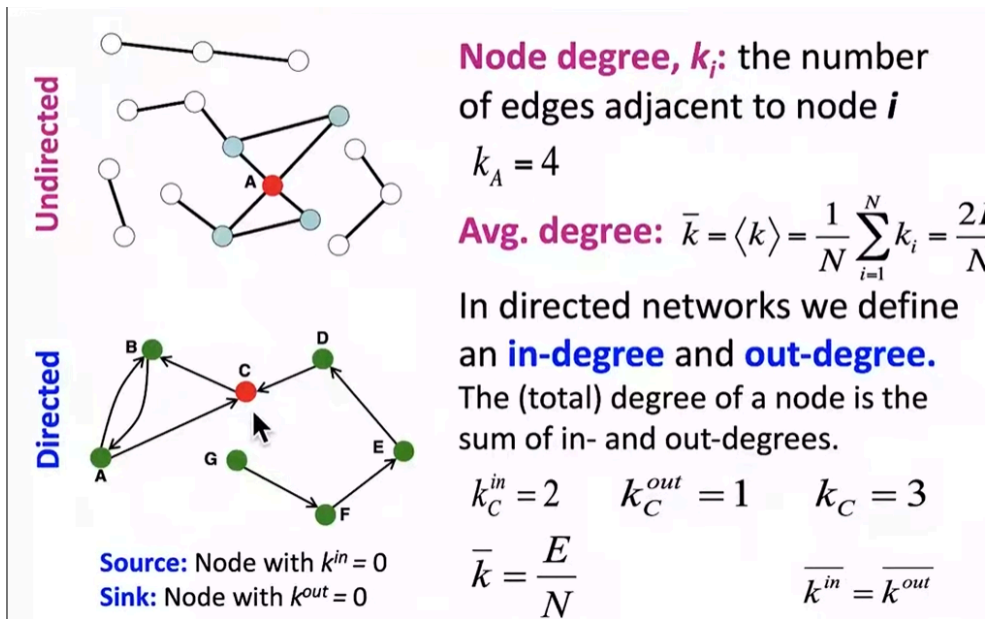
■ Examples:

- Phone calls
- Following on Twitter

- *Node Degree, k_i , the number of edges adjacent to node i
- *Average degree:*

$$\bar{k} = \langle k \rangle = \frac{1}{N} \sum_{i=1}^N k_i = \frac{2E}{N}$$

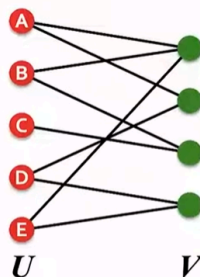
- In directed networks, we define an **in-degree** and **out-degree**. The total degree of a node is the sum of **in-degree** and **out-degree**.



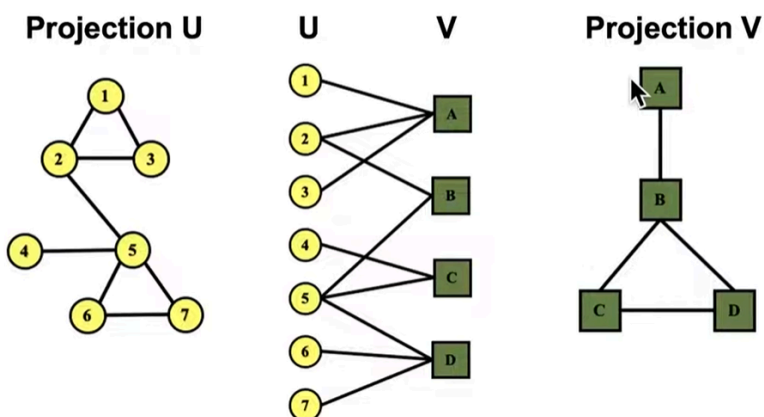
Bipartite Graph

Definition: the graph whose nodes can be divided into two disjoint sets U and V such that every link connects a node in U to one in V . (There is no internal connections)

- **Bipartite graph** is a graph whose nodes can be divided into two disjoint sets U and V such that every link connects a node in U to one in V ; that is, U and V are **independent sets**
- **Examples:**
 - Authors-to-Papers (they authored)
 - Actors-to-Movies (they appeared in)
 - Users-to-Movies (they rated)
 - Recipes-to-Ingredients (they contain)
- **"Folded" networks:**
 - Author collaboration networks
 - Movie co-rating networks



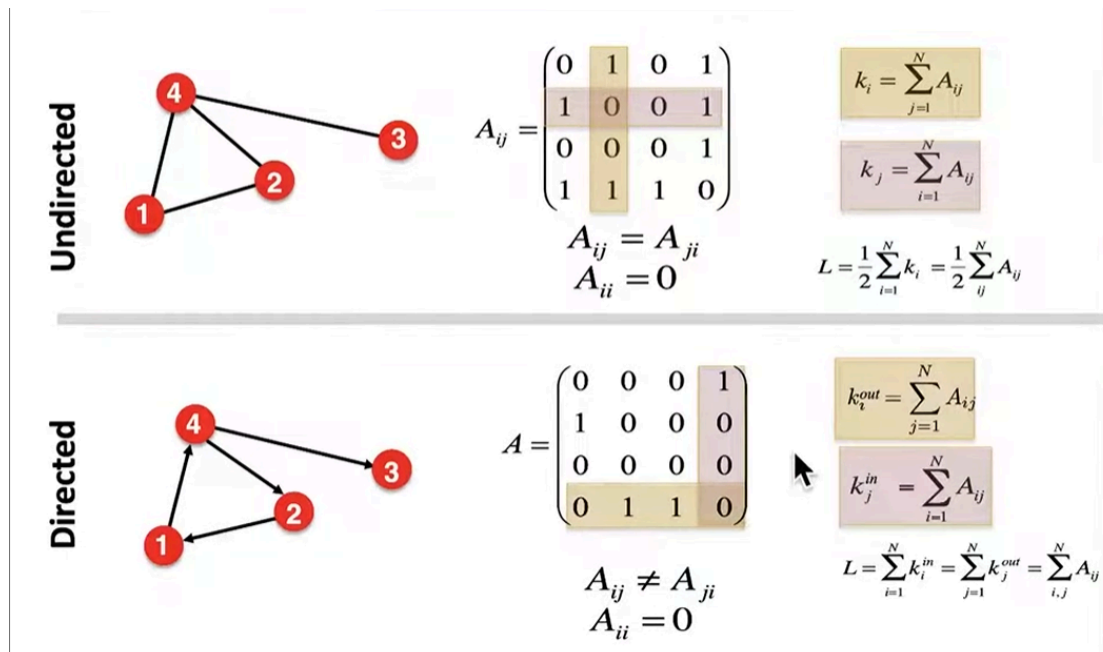
- Folded/Projected Bipartite graphs:
Where the Projection U represent the common connection to V , and vice versa



Adjacency

Adjacency Matrix

- 1 represents the connection
- For undirected graph, the matrix is naturally symmetric



Adjacency list

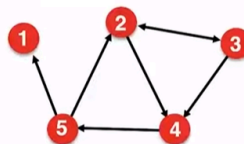
It is easier to work with if network is

- Large
- Sparse

It allows us to quickly retrieve all neighbors of a given node.

Adjacency list:

- Easier to work with if network is
 - Large
 - Sparse
- Allows us to quickly retrieve all neighbors of a given node
 - 1:
 - 2: 3, 4
 - 3: 2, 4
 - 4: 5
 - 5: 1, 2



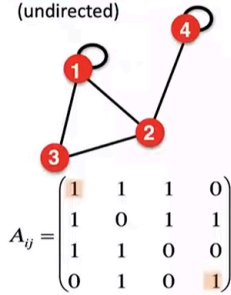
Node, Edge Attributes, and other graph types

- Weight, ranking, type, sign...
- Chemical, bonding types...
- Other graph types

Self-edges and Multigraphs

Self-edges (self-loops)

(undirected)

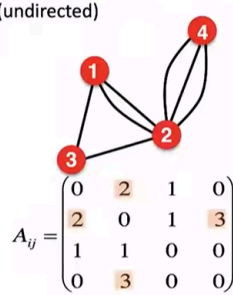


$$E = \frac{1}{2} \sum_{i,j=1, i \neq j}^N A_{ij} + \sum_{i=1}^N A_{ii}$$

Examples: Proteins, Hyperlinks

Multigraph

(undirected)



$$E = \frac{1}{2} \sum_{i,j=1}^N \text{nonzero}(A_{ij})$$

Examples: Communication, Collaboration

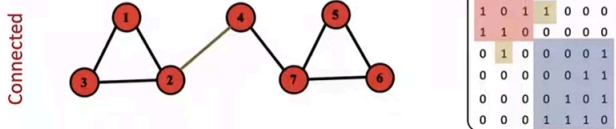
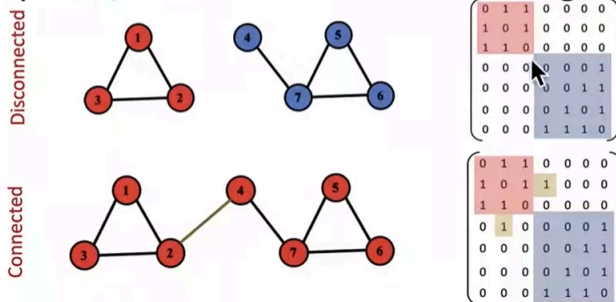
Connectivity

Connectivity: Any two vertices can be joined by a path

Disconnected: a graph that is made up by two or more connected components

- Representing in adjacency matrices:

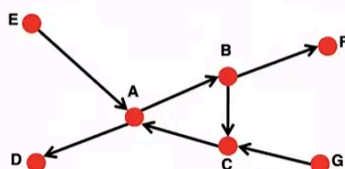
The adjacency matrix of a network with several components can be written in a block-diagonal form, so that nonzero elements are confined to squares, with all other elements being zero:



- Strongly and Weakly connected directly graph:

Strongly: has a path from each node to every other node and vice versa (A-B, B-A)

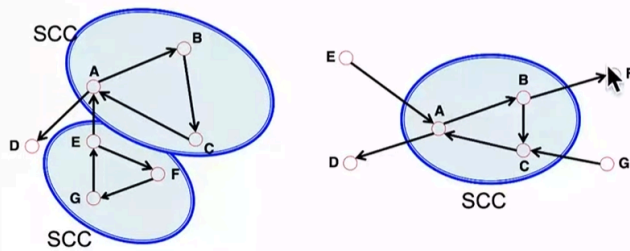
Weakly: is connected if we disregard the edge directions.



Graph on the left is connected but not strongly connected (e.g., there is no way to get from F to G by following the edge directions).

In this case, A-B is strongly connected, A-D and C-G is weakly connected. Furthermore, A-B-C shows a strongly connected component:

- **Strongly connected components (SCCs)** can be identified, but not every node is part of a nontrivial strongly connected component.



In-component: nodes that can reach the SCC,

Out-component: nodes that can be reached from the SCC.

Reference:

<https://www.youtube.com/watch?v=P-m1Qv6-8cl&list=PL-myaKI4DsIVPlos0HopgT8SjOn7HLJw8&index=24>

Lecture 1: Introduction to RL

Professor Emma Brunskill

CS234 RL

Winter 2026

- Today the 3rd part of the lecture includes some slides from David Silver's introduction to RL slides or modifications of those slides

